

CSCI 5540 – Fall 26

Tutorial 3

DENG Yangtao

Credits to UCSD CSE/DSC 291 Deep Learning Systems

What we will cover

1

ML compilation

Why automatic code generation is hard

2

Triton

A practical DSL for custom GPU kernels

3

Graph optimization

Rewrite graphs without changing outputs

4

Automation

From hand-written templates to verified rewrites

Main question: **how do ML systems get fast without hand-tuning everything?**

Quick GPU review: what matters for kernels?

Kernel

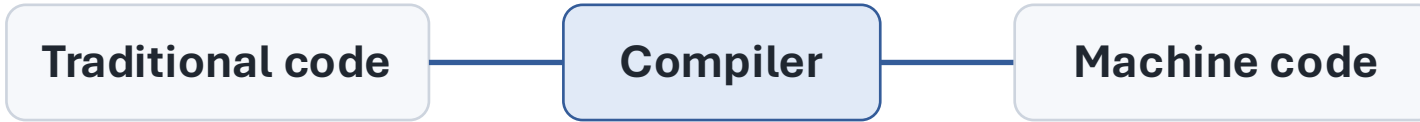
- A function executed by many GPU threads
- Programmer chooses grid/block mapping
- Performance depends on occupancy and memory behavior

Shared memory

- Fast memory shared inside one thread block
- Useful when multiple threads reuse the same data

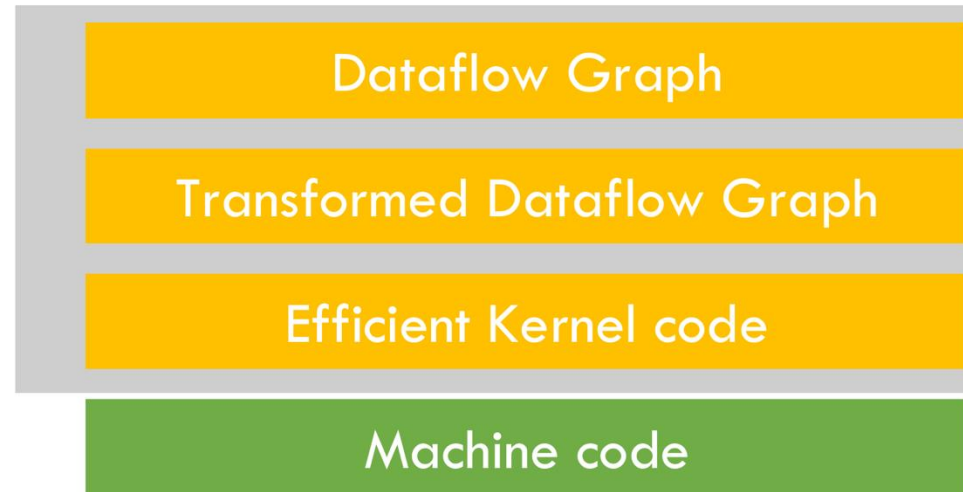
Enough parallel work
+
Coalesced memory access
+
Data reuse

ML programs have more optimization layers



ML systems add:

- Dataflow graph representation
- Graph transformations
- Operator/kernel generation



Three compiler problems in ML systems

Programming level

Translate user code into a compilable representation

Graph level

Rewrite the graph to reduce overhead

Operator level

Generate fast kernels for target hardware

Dataflow Graph

Transformed Dataflow Graph

Efficient Kernel code

Machine code

**Today's focus: operator level with Triton,
then graph-level transformations.**

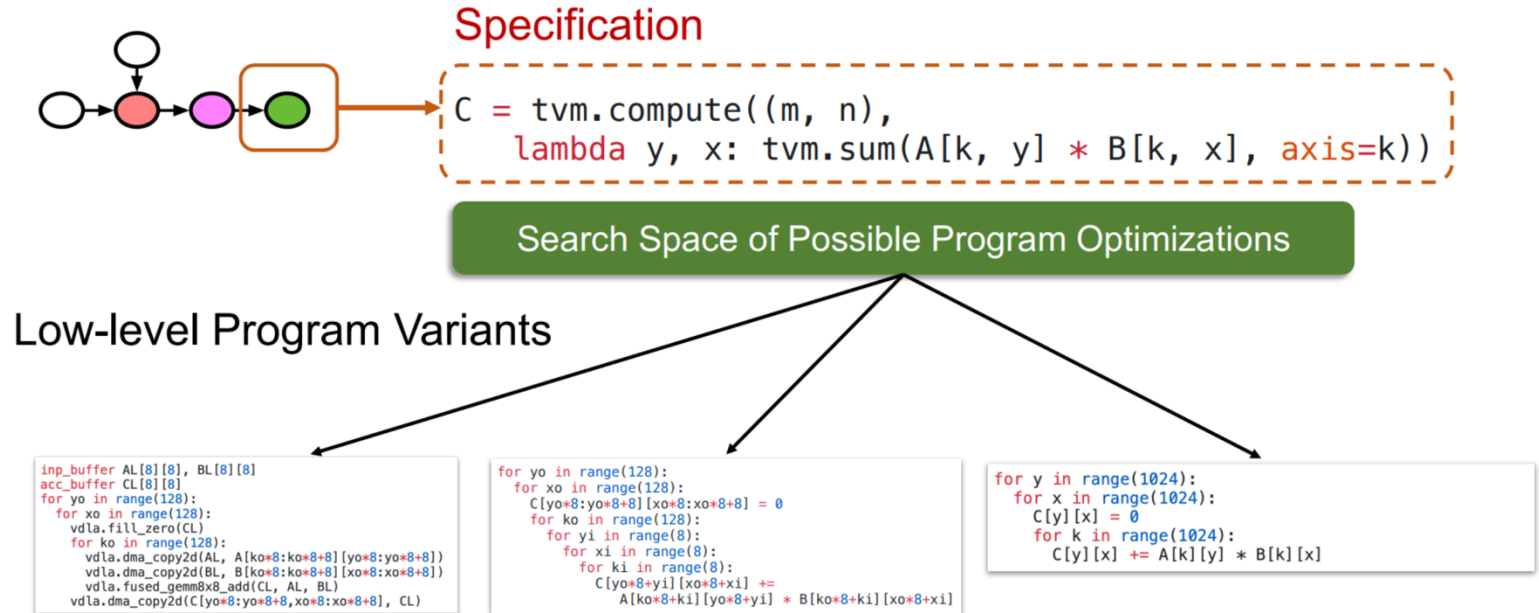
Operator compilation: many valid programs

One specification

$$C = A \times B$$

Many implementations

- Different loop orders
- Different tile sizes
- Different memory layouts
- Different thread/block mappings



The compiler searches this space.

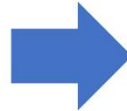
A basic schedule transform: loop splitting

Code

```
for x in range(128):  
    C[x] = A[x] + B[x]
```



```
for xo in range(32):  
    for xi in range(4):  
        C[xo * 4 + xi]  
        = A[xo * 4 + xi] + B[xo * 4 + xi]
```



```
def gpu_kernel():  
    C[threadId.x * 4 + blockIdx.x] = . . .
```



```
for xi in range(4):  
    for xo in range(32):  
        C[xo * 4 + xi]  
        = A[xo * 4 + xi] + B[xo * 4 + xi]
```

This creates structure that can map to blocks and threads.

Why Triton? A middle ground



More control

More automation

- Simpler than CUDA
- More expressive than graph-only compilers
- Still exposes program/block-level mapping

Triton programming model

@triton.jit

Kernels are defined in Python

tl.arange

Create vector offsets inside one program

tl.load / tl.store

Dereference pointer tensors with masks

tl.program_id

Map program instances to data blocks

Embedded in Python



Kernels are defined in Python using triton.jit

Pointer arithmetics



Users construct tensors of pointers and (de)reference them elementwise

Shape Constraints



Must have power-of-two number of elements along each dimension

Think in blocks of elements, not scalar GPU threads.

Example: elementwise add v2 ($z = x + y$)

Use multiple blocks

- Index the block and apply offset
- Adds bound check

```
import triton.language as tl
Import triton

@triton.jit
def _add(z_ptr, x_ptr, y_ptr, N):
    # same as torch.arange
    offsets = tl.arange(0, 1024)
    offsets += tl.program_id(0)*1024
    # create 1024 pointers to X, Y, Z
    x_ptrs = x_ptr + offsets
    y_ptrs = y_ptr + offsets
    z_ptrs = z_ptr + offsets
    # load 1024 elements of X, Y, Z
    x = tl.load(x_ptrs, mask=offset<N)
    y = tl.load(y_ptrs, mask=offset<N)
    # do computations
    z = x + y
    # write-back 1024 elements of X, Y, Z
    tl.store(z_ptrs, z)

N = 192311
x = torch.randn(N, device='cuda')
y = torch.randn(N, device='cuda')
z = torch.randn(N, device='cuda')
grid = (triton.cdiv(N, 1024), )
_add[grid](z, x, y, N)
```

When should we write a Triton kernel?

Good candidates

- The operator is memory-bound and has avoidable intermediate tensors
- A sequence of primitives can be fused into one pass
- The shape pattern is common enough to justify tuning

Less ideal

- The operator is already covered by cuBLAS or cuDNN
- The workload is too irregular for block-level vectorization
- The kernel is rarely called, so compile/tuning cost is not worth it

Rule of thumb

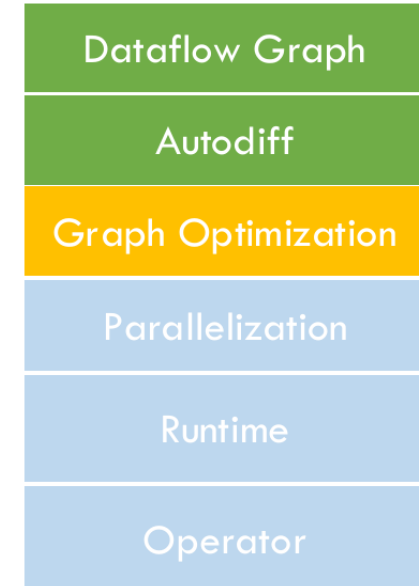
Use Triton when framework primitives are too coarse, but CUDA is more engineering than you need.

This is why fused attention, normalization, and custom reductions are common Triton targets.

Graph optimization: rewrite G into G'

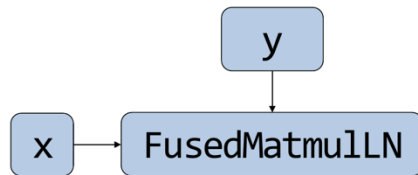
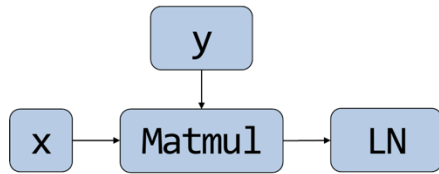


- Same output
- Lower runtime cost
- Usually implemented by pattern matching and replacement



Correctness is mandatory; speedup is hardware-dependent.

Template example: operator fusion



- Why operator fusion improves performance?
 - Reduce I/O
 - Reduce kernel launching
- Cons:
 - Requiring many fused ops: FusedABCOp
 - At some point, codebase becomes unmanageable

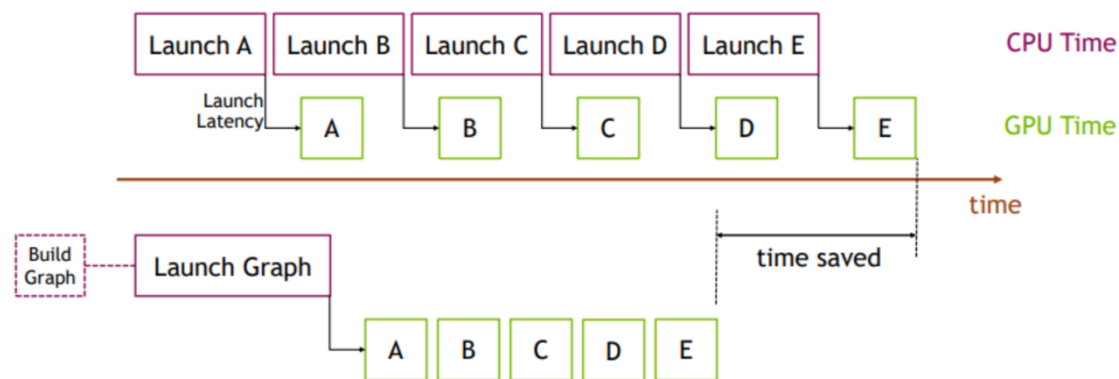
Practical graph capture: CUDA Graphs

Problem

- A model step may launch many small CUDA kernels
- CPU launch overhead and synchronization become visible

Idea

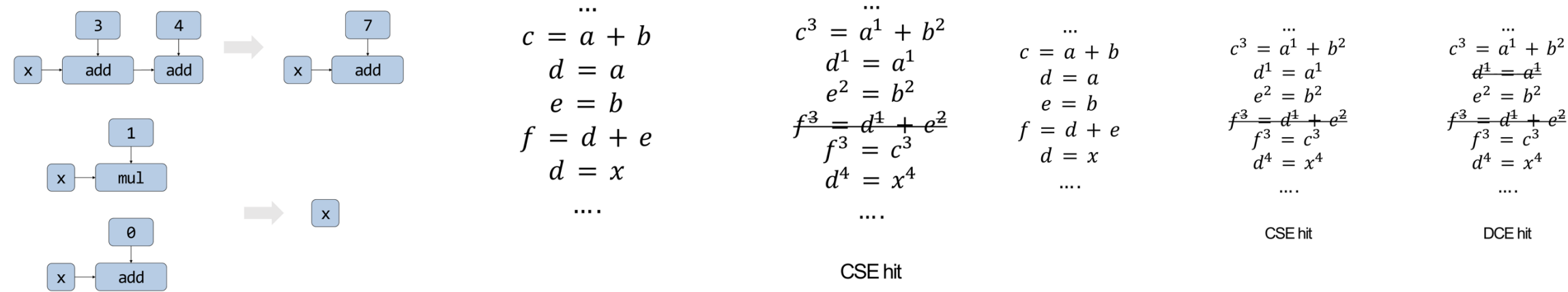
- Capture a fixed sequence of CUDA work
- Replay it with lower launch overhead
- Keep high-level code while optimizing execution



More reading: <https://pytorch.org/blog/accelerating-pytorch-with-cuda-graphs/>

This is graph-level optimization at the runtime boundary.

Simple graph rewrites: fold, reuse, delete



Why manual graph templates do not scale

Robustness

Heuristics do not apply to all DNNs or hardware

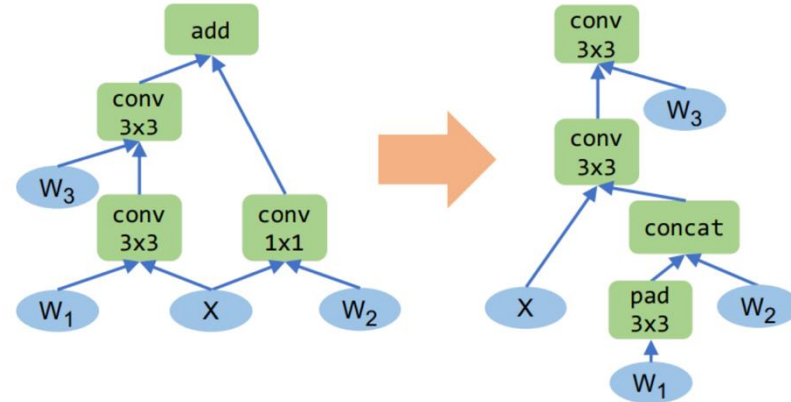
Scalability

New operators and graph structures need more rules

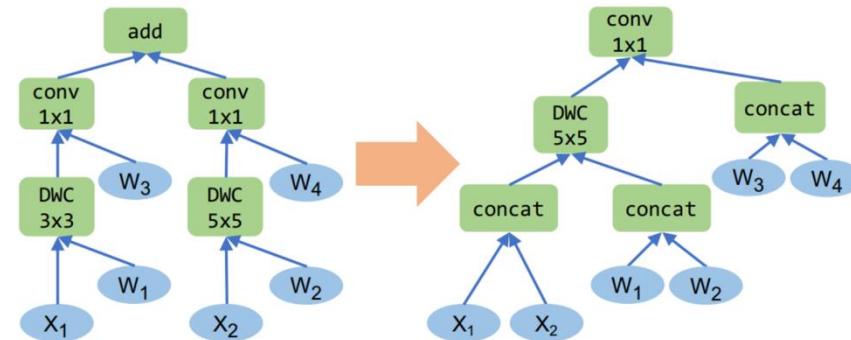
Performance

Subtle target-specific optimizations are easy to miss

Rules multiply across operators, graph structures, and hardware backends.



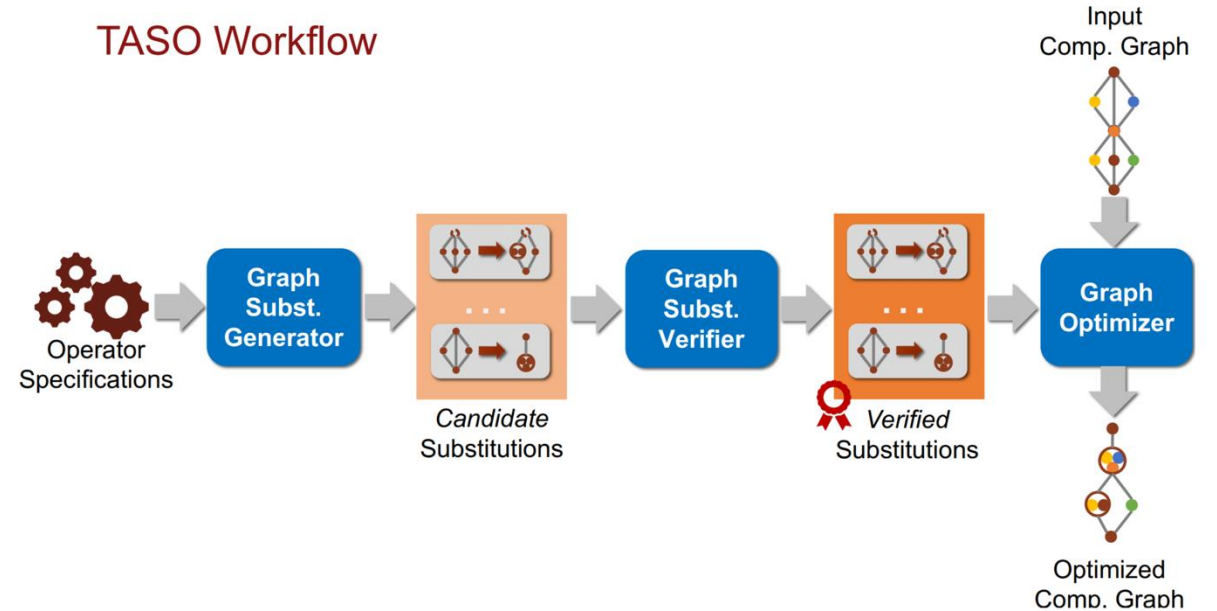
Only apply to **specific hardware**



Only apply to **specialized graph structures**

Automate graph substitutions

- 1 Operator specs** Describe tensor algebra semantics
- 2 Generate** Enumerate candidate substitutions
- 3 Verify** Keep only equivalent rewrites
- 4 Optimize** Apply verified rules to a graph



Takeaway: efficient ML systems combine operator kernels and graph-level rewrites.

Thanks!